Vaani

Voice-controlled laptop agent powered by Sarvam AI

Build window: 30 minutes | Platform: [Mac / Windows / Linux - fill in] | Language: Python 3.11+

> Paste this whole document into your coding agent. Model IDs and endpoints change, so every ID lives in **config.py** and **must be verified against docs.sarvam.ai** before running.

1. One-line pitch

Hold a hotkey, speak in any Indian language (or Hinglish), and an agent powered by Sarvam 105B operates your laptop. If something fails, it explains the problem back to you, out loud, in your own language.

2. Core flow

```
[Hold hotkey] -> record WAV
      |
      v
Sarvam STT-Translate (speech -> English text + detected language code)
      |
      v
Sarvam 105B Agent (tool-calling loop, max 6 steps)
   |- Laptop tools: open_app, open_url, search_web, type_text, press_key,
   |                hotkey, scroll, set_volume, screenshot, read_clipboard
   '- Sarvam tools: translate_text, read_screen (Sarvam Vision), speak (TTS)
      |
      v
Result or error -> translate to user's language -> Bulbul TTS -> play audio
```

3. Goals and non-goals

Goals (must ship)

- Push-to-talk voice capture.
- Speech to English command, with the user's language code retained.
- A 105B agent loop that calls laptop tools via function calling.
- Spoken replies (success and errors) in the user's original language.
- A live console or small window showing transcript, chosen action, and result.

Non-goals

Wake word, streaming audio, a polished GUI, multi-monitor support, and video dubbing (roadmap only).

4. Tech stack

- **Audio:** sounddevice + soundfile for recording; pynput for the hotkey; sounddevice or playsound for playback
- **API:** requests or the official sarvamai SDK
- **Laptop control:** pyautogui, subprocess, webbrowser, pyperclip
- **UI:** rich for the console
- **Secrets:** .env with SARVAM_API_KEY

5. Repo structure and interface contract

Each module exposes **exactly** the signatures below. This is what lets four people or agents build in parallel without blocking each other.

```
vaani/
|-- config.py        # models, keys, constants, SAFE_MODE
|-- audio_io.py      # Track A
|-- sarvam_client.py # Track B
|-- tools.py         # Track C (laptop tools + schemas)
|-- agent.py         # Track D (agent loop)
|-- main.py          # Track D (glue + hotkey loop + UI)
'-- .env
```

```
# audio_io.py
record_while_key_held(key="f9") -> str            # returns wav path
play_audio(path_or_bytes) -> None

# sarvam_client.py
stt_translate(wav_path) -> dict   # {"text_en", "lang": "te-IN", "raw_transcript"}
translate(text, src, tgt) -> str
tts(text, lang_code) -> bytes     # wav bytes, ready to play
chat(messages, tools=None) -> dict   # raw 105B response, OpenAI-style
vision_read(image_path, prompt) -> str

# tools.py
TOOL_SCHEMAS: list[dict]          # OpenAI-style function schemas
execute_tool(name, args) -> dict  # {"ok": bool, "result": str}  never raises

# agent.py
run_agent(command_en: str, lang: str) -> str   # final English message for user
```

6. Work split: 4 parallel tracks

Track	Owner	Builds	Done when
A: Audio	Agent / person 1	audio_io.py: hold F9 to record, release to save WAV (16 kHz mono), plus playback	Speak, get a WAV, play it back
B: Sarvam client	Agent / person 2	sarvam_client.py: STT-translate, translate, TTS, chat, vision wrappers with timeouts and one retry	Each function tested standalone with a sample input
C: Tools	Agent / person 3	tools.py: all laptop tools, schemas, whitelist, confirmation gate	execute_tool('open_app', {'name':'Chrome'}) works
D: Agent + glue	Agent / person 4	agent.py and main.py. Start with stubbed A/B/C, swap in real modules as they land	Typed command runs end to end with mocks

Timeline

Time	Activity
0:00 - 0:03	Scaffold the repo, config.py, and stub files with the signatures above
0:03 - 0:18	Tracks A to D run in parallel
0:18 - 0:25	Integrate: D replaces stubs with real modules
0:25 - 0:30	Test the 5 demo commands and fix bugs. Freeze at 0:30

7. Detailed requirements

7.1 Audio (Track A)

- Hold F9 to record, release to stop. Reject clips under 0.4s.
- Save to tmp/input.wav at 16 kHz mono.

7.2 Sarvam client (Track B)

- **STT + translate:** use the speech-to-text-translate endpoint (Saaras) so one call returns English text and detected language. If no language code is returned, fall back to plain STT (Saarika) with

language_code='unknown', then call translate.

- **TTS:** Bulbul, returning base64 audio decoded to bytes. Map language code to a valid speaker and language.
- **Chat:** Sarvam 105B via chat completions, passing tools if supported. **Fallback if native tool calling is unsupported:** prompt for strict JSON {"tool": "...", "args": {...}} or {"final": "..."} and parse it (strip code fences).
- **Vision:** accepts an image path and prompt, returns text. Used by read_screen.
- All calls: 20s timeout, one retry, errors raised as SarvamError(msg).

7.3 Tools (Track C)

Tool	Args	Notes
open_app	name	Mac: open -a; Win: start; Linux: xdg-open or the binary
open_url / search_web	url / query	webbrowser module
type_text	text	pyautogui.write, or clipboard paste for non-ASCII text
press_key / hotkey	key / keys[]	pyautogui
scroll	amount	positive up, negative down
set_volume	level 0-100	OS-specific
screenshot	none	saves tmp/screen.png, returns the path
read_screen	question	screenshot, then vision_read
translate_text	text, src, tgt	calls Sarvam translate
speak	text, lang	TTS and play
read_clipboard	none	returns text

Safety rules

- Tools are whitelisted; anything else returns {"ok": false}.
- No shell execution of model-generated strings. open_app takes a name only.
- SAFE_MODE=True: type_text that includes Enter, plus any send, delete, or purchase action, asks for a spoken or keyboard yes/no before running.
- pyautogui.FAILSAFE = True (mouse to a corner aborts everything).
- execute_tool never raises. It always returns an ok/result dict.

7.4 Agent (Track D)

System prompt (put in agent.py):

> *You are Vaani, an agent that controls the user's laptop (OS: {OS}). The user's command is in English, translated from {lang}. Use the provided tools to accomplish it, one step at a time. After each tool result, decide the next step. Use read_screen if you need to verify what's on screen. When done, reply with a short final message (under 20 words). If a tool fails twice, stop and explain the problem simply. Never invent tools.*

Loop: messages, then chat. If a tool call comes back, run execute_tool, append the result, and repeat. If a plain message comes back, that is the final answer. Cap at 6 iterations, then return "I couldn't finish this."

Error voice-back: any exception or failed final result becomes a friendly English sentence, translated to the user's lang, converted with tts, and played. If Sarvam itself is down, print the error and play a local beep.

7.5 Main loop

```
while True:
    wav = record_while_key_held("f9")
    r = stt_translate(wav)                        # show on screen
    msg = run_agent(r["text_en"], r["lang"])      # show each tool call live
    out = translate(msg, "en-IN", r["lang"]) if r["lang"] != "en-IN" else msg
    play_audio(tts(out, r["lang"]))
# wrap in try/except; on any failure run the error voice-back path
```

8. Demo script (acceptance tests)

#	Command	Expected
1	"Chrome kholo aur YouTube pe lo-fi music search karo"	Opens browser and searches
2	Same command in Telugu or Tamil	Same result, reply spoken in that language
3	"Notepad open chesi 'hello hackathon' ani type chey"	Opens app and types
4	"Screen pe emi undi cheppu"	Screenshot, vision, spoken description in user's language
5	"Open an app called Zxqw" (deliberate failure)	Spoken error in user's language

Success criteria: at least 4 of 5 pass, and end-to-end latency is under about 8s for simple commands.

9. Risks and fallbacks

Risk	Fallback
105B lacks native tool calling	JSON-in-prompt mode (see 7.2)
105B is slow	Switch the AGENT_MODEL constant to the smaller Sarvam chat model for the demo
Mic permissions (Mac)	Grant Terminal mic and Accessibility access in System Settings
pyautogui blocked	Enable Accessibility permissions; use pynput as a backup
Hotkey conflicts	Make PTT_KEY configurable

10. Roadmap (pitch only, not built)

- Live video dubbing with a floating overlay (system-audio capture, timed audio insertion, translated playback).
- Document and image translation via Sarvam Vision.
- Wake word and streaming STT.

Instructions to the coding agent

- Create the scaffold and stubs first (0:00-0:03), then build your assigned track.
- Do not change the contract signatures. If you must, say so loudly.
- Verify every Sarvam endpoint and model ID against docs.sarvam.ai before hardcoding it.
- Prefer working over pretty. Optimize for the 5-command demo.